# Lab_sheet-07

## Anish Tiwari

## Section→C

## Roll no.→11

## Experiment No. 7

### Experiment Name

Motion Estimation using Optical Flow Algorithms in Video Sequence

## Program Code and Output

### Step 1: Import Required Libraries

In [7]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Error: Camera not opened")
else:
    print("Camera opened successfully")
```

```
Error: Camera not opened
[ WARN:0@537.123] global cap_v4l.cpp:914 open VIDEOIO(V4L2:/dev/video0): can't open camera by index
[video4linux2,v4l2 @ 0x55b465e6ae40] ioctl(VIDIOC_G_INPUT): Inappropriate ioctl for device
[ERROR:0@537.124] global obsensor_uvc_stream_channel.cpp:163 getStreamChannelGroup Camera index out of range
```

### Step 2: Consecutive frames + Grayscale

In [ ]:
```
cap = cv2.VideoCapture(0)

ret1, frame1 = cap.read()
ret2, frame2 = cap.read()

if not ret1 or not ret2 or frame1 is None or frame2 is None:
    print("Camera frames could not be read. Check camera connection and permissions.")
else:
    gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

    cv2.imshow("Frame 1", gray1)
    cv2.imshow("Frame 2", gray2)

    cv2.waitKey(0)

cap.release()
cv2.destroyAllWindows()
```

**The Kernel crashed while executing code in the current cell or a previous cell.**

**Please review the code in the cell(s) to identify a possible cause of the failure.**

**Click <a href='https://aka.ms/vscodeJupyterKernelCrash'>here</a> for more info.**

**View Jupyter <a href='command:jupyter.viewOutput'>log</a> for further details.**

### Step 3: Lucas-Kanade Optical Flow

In [ ]:
```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)

ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

# Feature points select karna
p0 = cv2.goodFeaturesToTrack(
    old_gray,
    maxCorners=100,
    qualityLevel=0.3,
```

```
    minDistance=7
)

ret, frame = cap.read()
frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

# Lucas-Kanade Optical Flow
p1, status, error = cv2.calcOpticalFlowPyrLK(
    old_gray,
    frame_gray,
    p0,
    None
)

print("Original points:")
print(p0)

print("New points:")
print(p1)

cap.release()
```

```
Original points:
[[[297. 306.]]

 [[327. 252.]]

 [[  2. 388.]]

 [[320. 295.]]

 [[310. 217.]]

 [[ 70. 388.]]]
New points:
[[[296.78403   306.1164   ]]

 [[326.79022   252.00833  ]]

 [[  1.7269135 387.9735   ]]

 [[319.863     295.14728  ]]

 [[309.973     217.25887  ]]

 [[ 69.98818   388.09842  ]]]
```

## Step 4: Motion Vectors / Trajectories

In [ ]:

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)

ret, old_frame = cap.read()
if not ret or old_frame is None:
    cap.release()
    raise RuntimeError("Camera frame could not be read. Check camera connection and permissions.")

old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7
)

if p0 is None:
    cap.release()
    raise RuntimeError("No trackable features found in the first camera frame.")

while True:

    ret, frame = cap.read()

    if not ret or frame is None:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    p1, status, error = cv2.calcOpticalFlowPyrLK(
        old_gray,
        gray,
        p0,
        None
    )
```

```
    if p1 is not None and status is not None:

        good_new = p1[status == 1]
        good_old = p0[status == 1]

        for new, old in zip(good_new, good_old):

            x1, y1 = new.ravel()
            x2, y2 = old.ravel()

            x1, y1 = int(x1), int(y1)
            x2, y2 = int(x2), int(y2)

            cv2.arrowedLine(
                frame,
                (x2, y2),
                (x1, y1),
                (0, 255, 0),
                2
            )

        if len(good_new) == 0:
            break

        p0 = good_new.reshape(-1, 1, 2)

    old_gray = gray.copy()

    cv2.imshow("Motion Vectors", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

## Step 5: Farneback Dense Optical Flow

In [ ]:

```
import cv2

cap = cv2.VideoCapture(0)

ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

ret, frame = cap.read()
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

flow = cv2.calcOpticalFlowFarneback(
    old_gray,
    gray,
    None,
    0.5,
    3,
    15,
    3,
    5,
    1.2,
    0
)

print("Dense Optical Flow calculated")
print("Flow shape:", flow.shape)

cap.release()
```

## Step 6: Color-coded Dense Optical Flow

In [ ]:

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)

ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

while True:

    ret, frame = cap.read()

    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
    # Farneback
    flow = cv2.calcOpticalFlowFarneback(
        old_gray,
        gray,
        None,
        0.5,
        3,
        15,
        3,
        5,
        1.2,
        0
    )

    magnitude, angle = cv2.cartToPolar(
        flow[..., 0],
        flow[..., 1]
    )

    hsv = np.zeros_like(frame)

    hsv[..., 1] = 255

    # Direction
    hsv[..., 0] = angle * 180 / np.pi / 2

    # Magnitude
    hsv[..., 2] = cv2.normalize(
        magnitude,
        None,
        0,
        255,
        cv2.NORM_MINMAX
    )

    output = cv2.cvtColor(
        hsv,
        cv2.COLOR_HSV2BGR
    )

    cv2.imshow(
        "Farneback Dense Optical Flow",
        output
    )

    old_gray = gray.copy()

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

## Step 7: Lucas-Kanade vs Farneback Comparison

In [ ]:

```
import cv2
import time

cap = cv2.VideoCapture(0)

ret, frame1 = cap.read()
ret, frame2 = cap.read()

gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

# Lucas-Kanade
start = time.time()

p0 = cv2.goodFeaturesToTrack(
    gray1,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7
)

p1, status, error = cv2.calcOpticalFlowPyrLK(
    gray1,
    gray2,
    p0,
    None
)

lk_time = time.time() - start
```

```python
# Farneback
start = time.time()

flow = cv2.calcOpticalFlowFarneback(
    gray1,
    gray2,
    None,
    0.5,
    3,
    15,
    3,
    5,
    1.2,
    0
)

farneback_time = time.time() - start


print("Lucas-Kanade Time:", lk_time)
print("Farneback Time:", farneback_time)

cap.release()
```

Out[ ]: True

## Step 8: Object Speed / Lighting / Camera Motion

In [ ]:
```python
import cv2

cap = cv2.VideoCapture(0)

while True:

    ret, frame = cap.read()

    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Brightness check
    brightness = gray.mean()

    print("Average Brightness:", brightness)

    cv2.imshow("Video", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

## Step 9: Object Tracking / Surveillance

In [ ]:
```python
import cv2

cap = cv2.VideoCapture(0)

tracker = cv2.TrackerCSRT_create()

ret, frame = cap.read()

# Object select karo
bbox = cv2.selectROI(
    "Select Object",
    frame,
    False
)

tracker.init(frame, bbox)

while True:

    ret, frame = cap.read()

    if not ret:
        break

    success, bbox = tracker.update(frame)

    if success:

        x, y, w, h = map(int, bbox)
```

```
        cv2.rectangle(
            frame,
            (x, y),
            (x + w, y + h),
            (0, 255, 0),
            2
        )

        cv2.putText(
            frame,
            "Object Tracking",
            (x, y - 10),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.7,
            (0, 255, 0),
            2
        )

    cv2.imshow(
        "Surveillance / Object Tracking",
        frame
    )

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

### Step 10: Observations and Discussion

In [ ]:
```
print("OBSERVATIONS")
print("----------------------------")
print("1. Lucas-Kanade performs sparse optical flow.")
print("2. It tracks selected feature points.")
print("3. Farneback performs dense optical flow.")
print("4. Farneback estimates motion across the image.")
print("5. Object speed affects optical-flow accuracy.")
print("6. Lighting changes can affect motion estimation.")
print("7. Camera movement produces additional motion.")
print("8. Optical flow is useful for tracking and surveillance.")
```

```
OBSERVATIONS
----------------------------
1. Lucas-Kanade performs sparse optical flow.
2. It tracks selected feature points.
3. Farneback performs dense optical flow.
4. Farneback estimates motion across the image.
5. Object speed affects optical-flow accuracy.
6. Lighting changes can affect motion estimation.
7. Camera movement produces additional motion.
8. Optical flow is useful for tracking and surveillance.
```

## Viva / Theory Questions

### Q1. What is Optical Flow?

Optical Flow is the apparent motion of objects or pixels between two consecutive video frames. It is used for motion detection, tracking, surveillance, and scene understanding.

### Q2.Explain Lucas-Kanade Optical Flow.

Lucas-Kanade is a Sparse Optical Flow method. It tracks selected feature points between consecutive frames by analyzing their movement and image gradients.

### Q3. What is Dense Optical Flow?

Dense Optical Flow estimates motion for almost every pixel in an image, while Sparse Optical Flow estimates motion only for selected feature points.

### Q4. Lucas-Kanade vs Farneback

Lucas-Kanade: Sparse, tracks selected points and is computationally lighter.

Farneback: Dense, estimates motion for every pixel and requires more computation.

### Q5. Assumptions of Optical Flow

Main assumptions are:

Brightness remains approximately constant. Motion between frames is small. Nearby pixels have similar motion.

### Q6.Factors Affecting Accuracy

Accuracy can be affected by:

Object speed Lighting changes Camera motion Occlusion Motion blur Image noise

### Q7. Five Applications

Autonomous driving Video surveillance Robotics Gesture recognition Sports analytics

### Q8. EWhy Grayscale Images?

Grayscale images are generally used because optical flow mainly needs brightness/intensity information. It also reduces computational complexity.

### Q9. MLimitations of Optical Flow

**Optical flow can face problems with:**

Fast motion Changing illumination Occlusion Motion blur Textureless regions Camera movement

### Q10. Role in Autonomous Driving, Surveillance & Action Recognition

Optical flow provides motion information from video. It helps track objects in surveillance, understand movement in autonomous driving, and recognize human actions.